Assignment 3: SQL DDL and updates

Name: Arnav Singhal

MIS: 612203012

Div: 1; **Batch**: T1

Statement : Write the DDL and DML statements for the following.

- Each offering of a course (i.e. a section) can have many Teaching assistants; each teaching assistant is a student. Extend the existing schema(Add/Alter tables) to accommodate this requirement.

  Query:

```
CREATE TABLE section_TA (
    course_id   VARCHAR(8),
    sec_id      VARCHAR(8),
    semester    VARCHAR(6),
    year        NUMERIC(4,0),
    s_ID        VARCHAR(5),
    PRIMARY KEY (course_id, sec_id, semester, year, s_ID),
    FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES
section(course_id, sec_id, semester, year) ON DELETE CASCADE,
    FOREIGN KEY (s_ID) REFERENCES student(ID) ON DELETE CASCADE
);
```

According to the existing schema, one student can have only one advisor.

- Alter the schema to allow a student to have multiple advisors and make sure that you are able to insert multiple advisors for a student.

  Query:

```
ALTER TABLE advisor
DROP PRIMARY KEY,
ADD PRIMARY KEY (s_ID, i_ID);
```

Write SQL queries on the modified schema. You will need to insert data to ensure the query results are not empty.

Queries used to insert data:

```
INSERT INTO instructor VALUES
(99392, "Srinivas", "Physics", 90000.0),
(81293, "Ashok", "Comp. Sci.", 50000.0);

INSERT INTO advisor VALUES
(44553, 33456),
(44553, 99392),
(44553, 45565),
(44553, 81293),
(23121, 12121),
(23121, 83821),
(23121, 10101),
(76543, 99392),
(76543, 81293);
```

- Find all students who have more than 3 advisors

Query:

```
SELECT advisor.s_ID, student.name
FROM student INNER JOIN advisor
ON student.ID = advisor.s_ID
GROUP BY s_ID
HAVING COUNT(i_ID) > 3;
```

Output:

```
+-------+---------+
| s_ID  | name    |
+-------+---------+
| 23121 | Chavez  |
| 44553 | Peltier |
+-------+---------+
```

- Find all students who are co-advised by Prof. Srinivas and Prof. Ashok.

  Query:

```
SELECT DISTINCT s.name
FROM advisor AS a1
INNER JOIN advisor AS a2 ON a1.s_ID = a2.s_ID
INNER JOIN instructor AS i1 ON a1.i_ID = i1.ID
INNER JOIN instructor AS i2 ON a2.i_ID = i2.ID
INNER JOIN student AS s ON a1.s_ID = s.ID
WHERE i1.name = 'Srinivas' AND i2.name = 'Ashok';
```

  Output:

```
+---------+
| name    |
+---------+
| Peltier |
| Brown   |
+---------+
```

- Find students advised by instructors from different departments.

  Query:

```
SELECT DISTINCT s.name
FROM advisor AS a1
INNER JOIN advisor AS a2 ON a1.s_ID = a2.s_ID
INNER JOIN instructor AS i1 ON a1.i_ID = i1.ID
INNER JOIN instructor AS i2 ON a2.i_ID = i2.ID
INNER JOIN student AS s ON a1.s_ID = s.ID
WHERE i1.dept_name <> i2.dept_name;
```

Output:

```
+---------+
| name    |
+---------+
| Chavez  |
| Peltier |
| Brown   |
+---------+
```

Write SQL queries for the following:

- Delete the course CS 101. Any course which has CS 101 as a prereq should remove CS 101 from its prereq set. Create a cascade constraint to enforce the above rule, and verify that it is working.

  Query to alter foreign key constraint in prereq table:

  ```
  ALTER TABLE prereq
  DROP FOREIGN KEY prereq_ibfk_2,
  ADD CONSTRAINT fk_prereq
  FOREIGN KEY (prereq_id) REFERENCES course(course_id) ON DELETE
  CASCADE;
  ```

  Query:

  ```
  DELETE FROM course
  WHERE course_id = ‘CS-101’;
  ```

  Contents of relation prereq before execution of above query:

  ```
  +-----------+-----------+
  | course_id | prereq_id |
  +-----------+-----------+
  | BIO-301   | BIO-101   |
  | BIO-399   | BIO-101   |
  | CS-190    | CS-101    |
  | CS-315    | CS-101    |
  | CS-319    | CS-101    |
  | CS-347    | CS-101    |
  | EE-181    | PHY-101   |
  +-----------+-----------+
  ```

Contents of relation prereq after execution of query:

```
+-----------+-----------+
| course_id | prereq_id |
+-----------+-----------+
| BIO-301   | BIO-101   |
| BIO-399   | BIO-101   |
| EE-181    | PHY-101   |
+-----------+-----------+
```

- Delete all information in the database which is more than 10 years old. Add data as necessary to verify your query.

  Query:

  ```
  DELETE FROM section
  WHERE year < YEAR(CURDATE()) - 10;
  ```

  Result: All records in the relation section have been deleted as all course offerings were over 10 years old. As a result of the "ON DELETE CASCADE" clause, all records of the relations teaches, takes and section_TA have also been deleted.